

Q1 Copy Constructor and Deep Copy 10 marks	✓
Q2 Abstract Class and Runtime Polymorphism 10 marks	✓
Q3 Static Data Member 10 marks	✓
Q4 Optimize Student Average Calculation 10 marks	✓
Q5 Virtual Destructor and Memory Leak 10 marks	✓
Q6 Operator Overloading 10 marks	✓
Q7	✓

Q1 Copy Constructor and Deep Copy

10 marks

```
#include <iostream>
using namespace std;

class Number {
    int *value;
public:
    Number(int v) {
        value = new int(v);
    }

    void display() {
        cout << *value << endl;
    }
};

int main() {
    Number n1(100);
    Number n2 = n1;
    n1.display();
    n2.display();
    return 0;
}
```

Q7
Runtime Polymorphism
10 marks

Q8
Inheritance and Constructor
Order
10 marks

Q9
Encapsulation and Setter
Validation
10 marks

Q10
Constructor and Object
Initialization
10 marks

10/10 answered

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Student {
5     string name;
6     int marks;
7
8 public:
9     Student(string n, int m) {
10         name = n;
11         marks = m;
12     }
13
14     void display() {
15         cout << name << " " << marks << endl;
16     }
17 };
18
19 int main() {
20     string name;
21     int marks;
22
23     cin >> name >> marks;
24
25     Student s(name, marks);
26     s.display();
27 }
```

Input

Gopinath.s 95

Output



CO4_AT3_DEBUGGING

View Only

QUESTIONS

Q1

Copy Constructor and Deep Copy
10 marks



Q2

Abstract Class and Runtime Polymorphism
10 marks



Q3

Static Data Member
10 marks



Q4

Optimize Student Average Calculation
10 marks



Q5

Virtual Destructor and Memory Leak
10 marks



Q2 Abstract Class and Runtime Polymorphism

10 marks

```
#include <iostream>
using namespace std;

class Employee {
public:
    void calculateSalary() {
        cout << "Employee Salary" << endl;
    }
};

class FullTimeEmployee : public Employee {
    double salary;
public:
    FullTimeEmployee(double s) {
        salary = s;
    }

    void calculateSalary() {
        cout << salary << endl;
    }
};
```

← Back

```

4 class Student {
5 public:
6     int marks;
7
8     Student(int m) {
9         marks = m;
10    }
11 };
12
13 int main() {
14     int n;
15     cin >> n;
16
17     Student* students = new Student[5]{
18         Student(80),
19         Student(90),
20         Student(70),
21         Student(60),
22         Student(100)
23     };
24
25     int sum = 0;
26
27     for (int i = 0; i < n; i++) {
28         sum += students[i].marks;
29     }
30
31     cout << (double)sum / n << endl;
32

```

Output



QUESTIONS

Q1 Copy Constructor and Deep Copy 10 marks	✓
Q2 Abstract Class and Runtime Polymorphism 10 marks	✓
Q3 Static Data Member 10 marks	✓
Q4 Optimize Student Average Calculation 10 marks	✓
Q5 Virtual Destructor and Memory Leak 10 marks	✓
Q6 Operator Overloading 10 marks	✓
Q7	✓

Q3 Static Data Member

```
#include <iostream>
using namespace std;

class Employee {
    int count;
public:
    Employee() {
        count++;
    }

    void displayCount() {
        cout << count << endl;
    }
};

int main() {
    Employee e1;
    Employee e2;
    Employee e3;
    e3.displayCount();
    return 0;
}
```

our Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Number {
5     int *value;
6
7 public:
8     Number(int v) {
9         value = new int(v);
10    }
11
12    // Copy constructor - Deep Copy
13    Number(const Number &n) {
14        value = new int(*n.value);
15    }
16
17    void display() {
18        cout << *value << endl;
19    }
20
21    ~Number() {
22        delete value;
23    }
24 };
25
26 int main() {
```

Q4 Optimize Student Average Calculation

```
#include <iostream>
using namespace std;

class Student {
public:
    int marks;
    Student(int m) {
        marks = m;
    }
};

int main() {
    int n;
    cin >> n;

    Student *students = new Student[n]{
        Student(80), Student(90), Student(70), Student(60), Student(100)
    };

    for (int i = 0; i < n; i++) {
        int sum = 0;
        for (int j = 0; j <= i; j++) {
            sum += students[j].marks;
        }
    }
```

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Parent {
5 public:
6     virtual ~Parent() {
7         cout << "Parent Destructor" << endl;
8     }
9 };
10
11 class Child : public Parent {
12 public:
13     ~Child() {
14         cout << "Child Destructor" << endl;
15     }
16 };
17
18 int main() {
19     Parent *p = new Child();
20
21     delete p;
22
23     return 0;
24 }
```

Input

stdin input

Output

05 Virtual Destructor and Memory Leak

```
#include <iostream>
using namespace std;

class Parent {
public:
    ~Parent() {
        cout << "Parent Destructor" << endl;
    }
};

class Child : public Parent {
public:
    ~Child() {
        cout << "Child Destructor" << endl;
    }
};

int main() {
    Parent *p = new Child();
    delete p;
    return 0;
}
```

our Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Shape {
5 public:
6     virtual void area() {
7         cout << "Shape Area" << endl;
8     }
9 };
10
11 class Rectangle : public Shape {
12 public:
13     void area() override {
14         int l, b;
15         cin >> l >> b;
16         cout << l * b << endl;
17     }
18 };
19
20 class Circle : public Shape {
21 public:
22     void area() override {
23         int r;
24         cin >> r;
25         cout << 3.14 * r * r << endl;
26     }
27 }
```

Input

5 4
3

Output



```
Complex operator+(Complex c) {
```

```
Complex temp;
```

```
temp.real = real;
```

```
temp.imag = imag;
```

```
return temp;
```

```
}
```

```
void display0 {
```

```
cout << real << " + " << imag << "i" << endl;
```

```
}
```

```
};
```

```
int main0 {
```

```
int r1, i1, r2, i2;
```

```
cin >> r1 >> i1;
```

```
cin >> r2 >> i2;
```

```
Complex c1(r1, i1);
```

```
Complex c2(r2, i2);
```

```
Complex c3 = c1 + c2;
```

```
c3.display0;
```

```
return 0;
```

```
}
```



```

4 class Complex {
5     int real, imag;
6
7     public:
8         Complex(int r = 0, int i = 0) {
9             real = r;
10            imag = i;
11        }
12
13        Complex operator+(Complex c) {
14            Complex temp;
15            temp.real = real + c.real;
16            temp.imag = imag + c.imag;
17            return temp;
18        }
19
20        void display() {
21            cout << real << " + " << imag << "i" << endl;
22        }
23    };
24
25    int main() {
26        int r1, i1, r2, i2;
27
28        cin >> r1 >> i1;
29        cin >> r2 >> i2;
30
31        Complex c1(r1, i1);
32        Complex c2(r2, i2);

```

Q7 Runtime Polymorphism

```
#include <iostream>
using namespace std;

class Shape {
public:
    void area() {
        cout << "Shape Area" << endl;
    }
};

class Rectangle : public Shape {
public:
    void area() {
        int l, b;
        cin >> l >> b;
        cout << l * b << endl;
    }
};

class Circle : public Shape {
public:
    void area() {
        int r;
```

```

1 #include <iostream>
2 using namespace std;
3
4 class Vehicle {
5 public:
6     Vehicle() {
7         cout << "Vehicle Constructor" << endl;
8     }
9 };
10
11 class Car : public Vehicle {
12 public:
13     Car() {
14         cout << "Car Constructor" << endl;
15     }
16 };
17
18 int main() {
19     Car c;
20
21     return 0;
22 }

```

I

stdin input

Output

QUESTIONS

Q1

Copy Constructor and Deep Copy
10 marks



Q2

Abstract Class and Runtime Polymorphism
10 marks



Q3

Static Data Member
10 marks



Q4

Optimize Student Average Calculation
10 marks



Q5

Virtual Destructor and Memory Leak
10 marks



Q6

Operator Overloading
10 marks



Q7



Q8 Inheritance and Constructor Order

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    Vehicle0 {
        cout << "Vehicle Constructor" << endl;
    }
};
```

```
class Car : Vehicle {
public:
    Car0 {
        cout << "Car Constructor" << endl;
    }
};
```

```
int main0 {
    Car c;
    return 0;
}
```

Your Code

Input

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class BankAccount {
5     double balance;
6
7 public:
8     void setBalance(double b) {
9         if (b >= 0)
10             balance = b;
11         else
12             balance = 0;
13     }
14
15     void deposit(double amount) {
16         if (amount > 0)
17             balance += amount;
18     }
19
20     void withdraw(double amount) {
21         if (amount > 0 && amount <= balance)
22             balance -= amount;
23     }
24
25     void display() {
26         cout << balance << endl;
27     }
28 }
```

Input

1000 500 200

Output



QUESTIONS

Q1

Copy Constructor and Deep Copy
10 marks



Q2

Abstract Class and Runtime Polymorphism
10 marks



Q3

Static Data Member
10 marks



Q4

Optimize Student Average Calculation
10 marks



Q5

Virtual Destructor and Memory Leak
10 marks



Q6

Operator Overloading
10 marks



Q7



Q9 Encapsulation and Setter Validation

```
#include <iostream>
using namespace std;
```

```
class BankAccount {
```

```
public:
```

```
double balance;
```

```
void deposit(double amount) {
```

```
    balance += amount;
```

```
}
```

```
void withdraw(double amount) {
```

```
    balance -= amount;
```

```
}
```

```
void display() {
```

```
    cout << balance << endl;
```

```
}
```

```
};
```

```
int main() {
```

```
    BankAccount account;
```

```
    double initial, deposit, withdraw;
```

```
    cin >> initial >> deposit >> withdraw;
```

```
    account.balance = initial;
```

```

1 #include <iostream>
2 using namespace std;
3
4 class Employee {
5     static int count;
6
7 public:
8     Employee() {
9         count++;
10    }
11
12    void displayCount() {
13        cout << count << endl;
14    }
15 };
16
17 // Initialize static data member
18 int Employee::count = 0;
19
20 int main() {
21     Employee e1;
22     Employee e2;
23     Employee e3;
24
25     e3.displayCount();
26
27     return 0;

```

stdin input

Output

Q10 Constructor and Object Initialization

```
#include <iostream>
using namespace std;

class Student {
    string name;
    int marks;
public:
    Student(string n, int m) {
        n = name;
        m = marks;
    }

    void display0 {
        cout << name << " " << marks << endl;
    }
};

int main0 {
    string name;
    int marks;
```



Output

```
1 public:
2     virtual void calculatesalary() = 0;
3
4     virtual ~Employee() {}
5 };
6
7 class FullTimeEmployee : public Employee {
8     double salary;
9
10 public:
11     FullTimeEmployee(double s) {
12         salary = s;
13     }
14
15     void calculatesalary() override {
16         cout << salary << endl;
17     }
18
19 };
20
21
22
23
24 class PartTimeEmployee : public Employee {
25     int hours;
26     double rate;
27
28 public:
29     PartTimeEmployee(int h, double r) {
30         hours = h;
31         rate = r;
32     }
33 }
```